Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now

Mip + Python + Optimization + School Timetable +

School scheduling with MIP

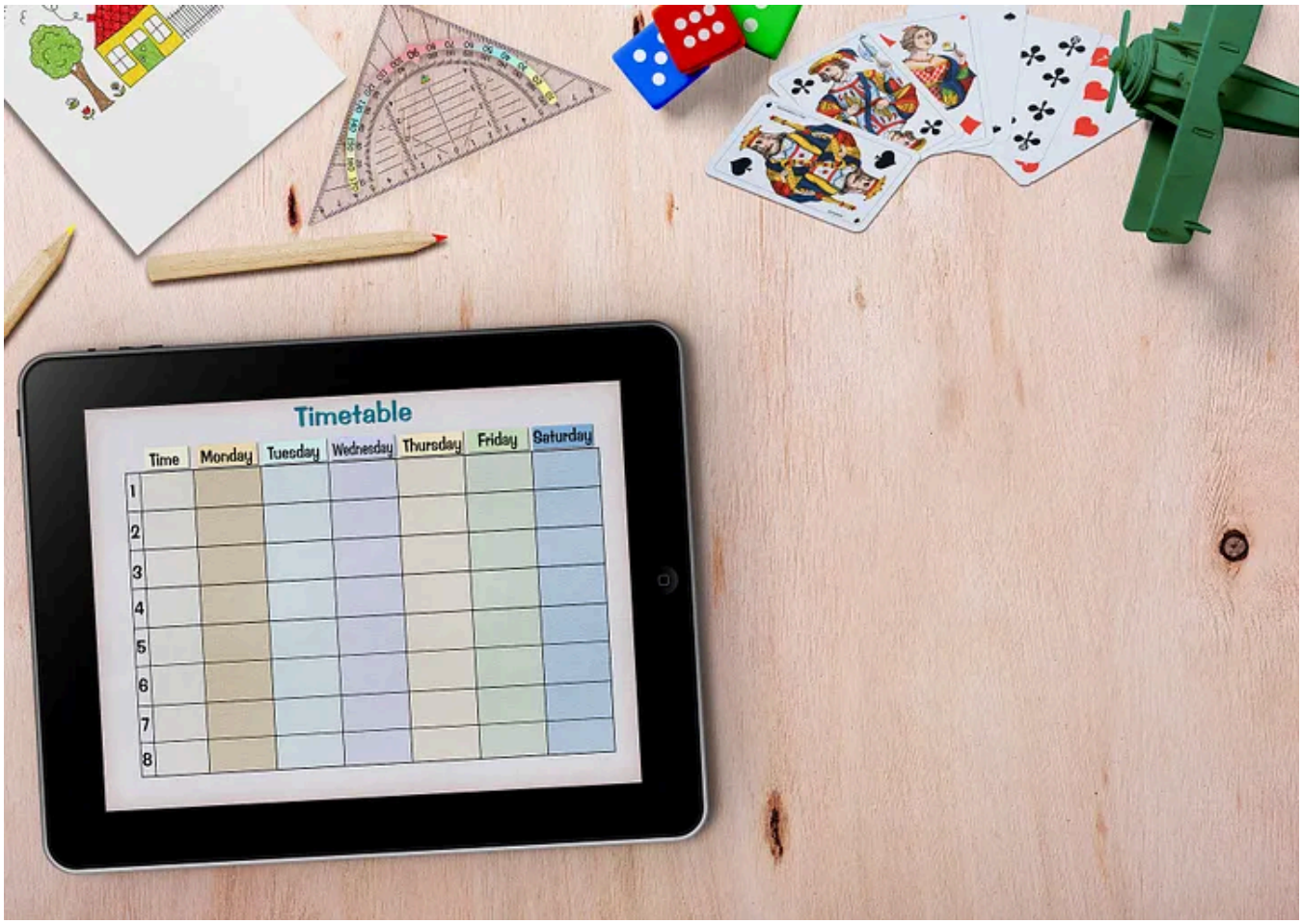
10 min read · Jan 23, 2022



Gianfranco Tavella Follow

Listen Share ••• More

An application of MIP at classic school scheduling problem



https://www.maxpixel.net/Timetable-Tablet-Drawing-Pad-Colored-Pencils-3222252

Introduction

The problem of how to assign teachers to classes, taking into account the innumerable demands and needs of each teacher, class, school system has haunted me ever since I was in high school.

At the time I saw my math teacher with a huge cardboard, a pencil and an eraser, trying all the possible combinations by hand until he found the one that satisfied him. Of course the solution was subject to compromise and modification and I always saw him suffer. But he succeeded. I, with my commodore 64 and the BASIC language could not find a way to solve the problem which, as I will discover only after many years, unfortunately is NP-Complete, i.e. the number of combinations grows exponentially with the increase of the variables, coming very quickly to not being able to be solved with methods such as brute force that is by determining all the possible combinations and evaluating their performance.

But, I repeat, he could do it by hand, so now with the computing power we have available, I tried to solve the problem with a MIP approach, that is, using Mixed Integer Programming techniques.

It is a descriptive system that uses mathematical techniques to find an optimal or feasible solution to a problem modeled according to MIP logics.

In this case, it is essential to identify the decision-making variable in the best way, according to the approach one wants to attempt; in my case I thought of a binary variable with a series of indices better explained later but which in practice tell me if a certain professor teaches a certain subject, in a certain class at a certain time of day. Then it is necessary to define a series of constraints to limit the field of possible solutions, such as, for example, a teacher cannot teach at the same time in two classes, or that one class is busy all hours. We will then see further constraints. The last piece is the definition of the objective function that allows me to indicate on the basis of what I consider better one solution than another. For example, a parameter could be that a teacher teaches the minimum possible number of subjects so that he can focus only on those, or as few classes as possible so that he does not have to remember too many student. Unfortunately, in this case I couldn't find any linear expressions that would allow me to set the object function, so I didn't really set it. I hope someone would like to team to complete this project in MIP, if even possible.

. . .

Decision Variable

As said, I choice the more granular solution for the decision variable using different indexes. One for each field of interest.

$$\textbf{Decision Variable } x_{tgmdh}$$

Where indexes are:

- teacher **t**
- grade **g**
- matter **m**
- day of the week **d**
- hour of the day **h**

In code:

```
self.x = {
 (t, g, m, d, h): self.model.add_var(
 name="x({},{},{},{},{})".format(t, g, m, d, h),
 var_type=BINARY
 )
 for t in self.teacher
 for g in self.grade
 for m in self.matter
 for d in self.day
 for h in self.hour
 if self.check(t, g, m, d)
 }
```

$$x_{Adam,1B,English,Friday,12:30-13:30} = 1$$

This means that teacher Adam for grade 1B teach English Friday from 12:30 to 13:30 (5th hour). Of course if 1 means he/she works, 0 means that don't teach. The self.check function take care of the fact that some combinations do not are valid. By example, some teacher do not teach some matters and so no. This help to reduce the number of variables. And you will see pratically in every contraint to avoid error for combination that don't exist.

. . .

COSTRAINTS

Single Constraint

This constraint is useful when we want to assure that a teacher teach exactly that matter, that hour of the day in that grade. In this case the variable is set to 1. Instead, on the contrary, if we want to be sure that a teacher doesn't teach exactly that matter, that hour of the day in that grade, we set the variable to zero.

$$x_{t,g,m,d,h} = 1$$

$$x_{t,g,m,d,h} = 0$$

```
def addConstraintSingle(self, t, g, m, d, h, forcing):
        '''
        Single constraint to set a particular teacher teach a
particolar matter in a particular grade, day, hour. Or viceversa.
        '''
        if self.check(t, g, m, d):
            if forcing=="Force":
                self.model += self.x[t, g, m, d, h] >= 1.0
            if forcing=="Forbid":
                self.model += self.x[t, g, m, d, h] <= 0.9
```

Costraint Min Max Hours for each Teacher

Each teacher must teach at least the minimum of hours a week and not more than a max of hours prefixed and settled for each teacher.

We can translate this concept in formula assuring that for each teacher the grand sum of x for every grade, matter, day of the week and hour is not minus than the minimum and not max the maximum: e.g. min 16 max 24.

$$\forall \text{ teacher}: \sum_{grade,matter,day,hour} x_{t,g,m,d,h} >= \text{teacher[t]["minWeekHours"]}$$

$$\sum_{grade,matter,day,hour} x_{t,g,m,d,h} <= \text{teacher[t]["maxWeekHours"]}$$

in code:

```
def addConstraintMinMaxHoursForTeacher(self):
        '''
        Constraint for assure that work hours for week for each
teacher are in range Min Max Hours for week for teacher
        '''
        for t in self.teacher:
            self.model += xsum(self.x[t, g, m, d, h]
                           for g in self.grade
                           for m in self.matter
                           for d in self.day
                           for h in self.hour
                           if self.check(t, g, m, d) ) >=
self.teacher[t]['minWeekHours']
            self.model += xsum(self.x[t, g, m, d, h]
                           for g in self.grade
                           for m in self.matter
                           for d in self.day
                           for h in self.hour
                           if self.check(t, g, m, d) ) <=
self.teacher[t]['maxWeekHours']
```

Constraint each grade works all days

Each grade must work for all the hours of each day

For each grade, for each day, for each hour, the grand sum of every teacher, matter must be exactly 1.

$$\forall \text{ grade, day, hour}: \sum_{teacher,matter} x_{t,g,m,d,h} == 1$$

In code:

```
def addConstraintEachHourBusyForEachGrade(self):
        '''
        Constraint for assure that each hour in each grade is busy
        '''
        for g in self.grade:
            for d in self.day:
                for h in self.hour:
                    self.model += xsum(self.x[t, g, m, d, h]
                                       for t in self.teacher
                                       for m in self.matter
                                       if self.check(t, g, m, d) ) == 1.0
```

Constraint that each grade have the right amount of hours of each matter
Each grade have a certain number of hours for matter that must be teached.
For each grade and for each matter, the grand sum over x for teacher, day, hour must be the indicated number of hours.

$$\forall \text{ grade, matter if grade, matter in hoursForMatterForGrade[]: } \sum_{teacher,day,hour} x_{t,g,m,d,h} == \text{hoursForMatterForGrade[g,m]}$$

In code:

```
def addConstraintHoursForMatterForGrade(self):
        '''
        Constraint to assure that the exact amount of hours of
matter for each grade is teached
        '''
        for g in self.grade:
            for m in self.matter:
                if (g,m) in self.hoursForMatterForGrade:
                    self.model += xsum(self.x[t, g, m, d, h]
                                       for t in self.teacher
                                       for d in self.day
                                       for h in self.hour
                                       if self.check(t, g, m, d) ) ==
self.hoursForMatterForGrade[g,m]
```

Constraint that a teacher can’t work at same time in two different grades
A teacher can’t teach a matter in more than one grade the same day, hour.
For each teacher, day, hour the grand sum of x for over the grade and the matter can’t be more than 1.

$$\forall \text{ teacher, day, hour} \quad \sum_{grade,matter} x_{t,g,m,d,h} <= 1.0$$

In code:

```
def addConstraintTeacherTeachInOnlyOneGradeAtSameTime(self):
        '''
        Constraint to assure that a teacher can't teach in different
class at the same time, day, hour
        '''
        for t in self.teacher:
            for d in self.day:
                for h in self.hour:
                    self.model += xsum(self.x[t, g, m, d, h]
                                       for g in self.grade
                                       for m in self.matter
                                       if self.check(t, g, m, d) ) <= 1.0
```

Constraint that assure consecutive hours for exam

For some matters needs to have a certain amount of consecutives hours (e.g. 2) for written exam.

It's hard to set it as a soft constraint (e.g. in the objective function) so I set it as and hard constraint. That means I cast this hours in the decision variable and let the MIP engine to found a solution (if any) using this hard constraint like a pivot. If not found a solution we need to set differently this contraint.



In code:

```
def addConstraintSetGradeMatterDayHour(self):
        '''
        Constraint to assure consecutive hours for exam
        '''
```

```
        def setGradeMatterDayHour(g, m, d, h, v):
            self.model += xsum(self.x[t, g, m, d, h]
                               for t in self.teacher
                               if self.check(t, g, m, d) ) == v

        hours = list(self.hour)
        for g,m in self.consecutiveHours:
            p = hours.index(self.consecutiveHours[g,m]
['hourToStart'])
            d = self.consecutiveHours[g,m]['day']
            for i in range(0,len(hours)):
                if i in range( p, p + self.consecutiveHours[g,m]
['hours'] ):
                    setGradeMatterDayHour(g, m, d, hours[i], 1.0)
                else:
                    setGradeMatterDayHour(g, m, d, hours[i], 0.0)
```

. . .

Objective function

To obtain a good solution, we have different aspect that we need to take care of, but unfortunately I was not able to found a solution using a Linear Model:

- Difference among the real number of hours that a teacher work a week ought to be similar to the normal number of hours a teacher ought to work (needs an abs() function that's not linear)
- The same matter in the same grade must/ought to be teached from the same teacher
- The grades number where a teacher work ought to be minimized (e.g. a teacher prefer not to have a lot of grades)
- The teachers number for each grade ought to be minimized (e.g. students prefer not to have a lot of different teachers)
- The matters number teached by each teacher ought to be minimized (e.g. teachers prefer not to teach a lot of different matters)
- Some teacher could choice not to teach first hours or last hours of the day for familiar or transportation issues

- Some matters is better to be set first hours (e.g. physical education) or last hours of the day (e.g. Religion so ateist could exit before last hour) for some other matters is better to be not teached at first or last hours (e.g. mathematics because of exam)

- Each teacher ought to work for a comparable number of hours for each day of the week in which he/she work (e.g. not 1 hour monday and 5 hours tuesday. Better 3 hours monday and 3 hours tuesday)

- Each teacher ought to have the minimum number of hole hours (free hours between teached hours) in a week, but at least a prefixed number (for parents meeting) (e.g. teachers desire not have free hours between two teaching hours but however they need to have at least 1 hour a week to meet the student's parents)

The problem is that for some ways or the other, each of this aspects need a Non-linear function, and this is not possible with MIP python. Or at least i don't know any way, and I'm more than happy to listen to suggestion.

. . .

Data

Data are in a separate class **TeacherClassSchedulingData** to separe data from code. I don't discuss here because are really simple and direct Python dictionaries. Just I say that I used Dictionary instead of list because of extendibility. By example, I could have used List for grades instead of Dictionary but in this way I can extend easily the data relatives to grades (e.g. I can take care of the fact that grades are in different building so teachers can't teach in grades that are in different building in consecutives hours because they need to have the time to walk in)

. . .

Results

Using data in the example class, you will see the number of decision variables and constraints created. Interesting to note is the strong reduction of the decision variables number.

```
Variables x created: 10770 instead of 28350
Constraints created: 902
Feasible Solution Found ! 1
```

Finally, we have data visualization, two different cardboards, one for grades and the other for teacher. Note that “Adam” have two free day based on the data desiderata.

Grade: 1A	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:30-09:30	Italian Adam	English Ida	Latin Adam	History Adam	Italian Ida	English George
09:30-10:30	History Eve	Italian Adam	Physical Educat	Physical Educat	History Frank	Mathematic John
10:30-11:30	Latin Adam	Religion Frank	Italian Frank	Italian Adam	English Ida	Mathematic Char
11:30-12:30	Mathematic Davi	English Bob	Latin Adam	Italian Adam	English David	English Ida
12:30-13:30	Mathematic Davi	Latin Adam	Mathematic Char	History David	Mathematic John	Religion Charli

Grade: 1B	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:30-09:30	Latin Frank	Italian Adam	Italian Ida	Italian Bob	English Bob	Mathematic Ida
09:30-10:30	History Adam	Latin Frank	Mathematic Char	Italian Adam	English Eve	Religion Charli
10:30-11:30	Mathematic Fran	Latin Adam	Latin George	History Charlie	Religion Charli	Italian Ida
11:30-12:30	Italian Adam	Physical Educat	English John	Mathematic Char	English John	Mathematic Geor
12:30-13:30	History Adam	Physical Educat	English Ida	History Adam	English Ida	Mathematic Fran

Scheduling “cardboard” visualization for grades “1A” e “1B”. Note for grade 1A that “Adam” teach Italian Monday first hour but not second;Mathematic is teached at last two hours of Monday and English Friday at 3th and 4th hours for exam as desiderata

Adam	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:30-09:30	1A Italian	1B Italian	1A Latin	1A History		
09:30-10:30	1B History	1A Italian		1B Italian		
10:30-11:30	1A Latin	1B Latin		1A Italian		
11:30-12:30	1B Italian		1A Latin	1A Italian		
12:30-13:30	1B History	1A Latin		1B History		

John	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:30-09:30			2A Italian		2B History	2B Mathematic
09:30-10:30	2A Physical Edu		1A Physical Edu	1A Physical Edu	2A Italian	1A Mathematic
10:30-11:30	2C Physical Edu		2C English		2C Italian	1C Latin
11:30-12:30	1C Mathematic		1B English		1B English	2B English
12:30-13:30	2A History		2C Italian	1C Italian	1A Mathematic	1C Italian

Henry	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
08:30-09:30	2B Mathematic			2A Latin	1C Latin	
09:30-10:30	2B Italian	2A English	2A Italian	1C Mathematic	1C Latin	
10:30-11:30	1C Mathematic		2A Mathematic			
11:30-12:30		2C History	2A Latin	1C English		
12:30-13:30	2B Mathematic	2B Italian	2B History			

Scheduling “cardboard” for teachers “Adam” “John” and “Henry” with evidence of free days as request in desiderata (e.g. Adam have two free days and others only one as desiderata)

• • •

Code

All code is on GitHub https://github.com/gt1967/MIP-School-Timetabling

. . .

Conclusion

This works. And really fast. Matters of seconds also for 10k decision variables. Try by yourself. But there are some limitations mainly in the objective function because of the circumstance that the same matter could be teached by different teachers (e.g. english teached from two different teachers in the same grade). In real life this could be resolved strongly limitating matter and class for each teacher. E.g. the principal decide who teach what in each class.

Another point that I would like to point out is that the model could easily extend. If, by example, we must take care of classrooms we can simply add another index **c classroom** and modify in accord the constraints. Easy and painless.

Probably this problem needs to be taken like a non linear problem. Maybe Pyomo could be usefull ? Stay tuned.

Mip + Python + Optimization + School Timetable +



Follow

Written by Gianfranco Tavella

0 followers · 1 following

No responses yet